POLITECHNIKA MORSKA W SZCZECINIE

JEDNOSTKA ORGANIZACYJNA:
WYDZIAŁ MECHATRONIKI I ELEKTROTECHNIKI
KATEDRA ROBOTYKI I STEROWANIA

INSTRUKCJA

Tworzenie programu, instrukcje warunkowe Podstawy Pythona (Laboratorium L1)

Opracował:	dr inż. Łukasz Mozga / mgr inż. Artur Górnostaj
Zatwierdził:	dr inż. Ewelina Kostecka
Obowiązuje od:	2026/2027

Politechnika Morska w Szczecinie. Wszelkie prawa autorskie zastrzeżone.

SPIS TREŚCI

1. **CEL I ZAKRES ĆWICZENIA**
2. **CZĘŚĆ TEORETYCZNA**
   - 2.1. Pierwszy program w języku Python
   - 2.2. Zmienne i podstawowe typy danych
   - 2.3. Zasady nazywania zmiennych
   - 2.4. Funkcja **print()**
   - 2.5. Pobieranie danych za pomocą **input()**
   - 2.6. Konwersja typów danych
   - 2.7. Podstawowe operatory arytmetyczne
   - 2.8. Operatory porównania
   - 2.9. Operatory logiczne
   - 2.10. Instrukcja warunkowa if
   - 2.11. Instrukcja **if...else**
   - 2.12. Instrukcja **if...elif...else**
   - 2.13. Wcięcia w języku Python
   - 2.14. Najczęściej popełniane błędy
3. **PRZEBIEG ĆWICZENIA**
4. **WARUNKI ZALICZENIA**
5. **EFEKTY KSZTAŁCENIA**
6. **LITERATURA**

Politechnika Morska w Szczecinie. Wszelkie prawa autorskie zastrzeżone.

1. CEL I ZAKRES ĆWICZENIA

Celem laboratorium jest zapoznanie studentów z podstawami tworzenia i uruchamiania programów w języku Python oraz wykorzystaniem instrukcji warunkowych do realizacji prostych algorytmów decyzyjnych.

W ramach ćwiczenia student powinien opanować:

- tworzenie i uruchamianie prostych programów w języku Python,
- stosowanie komentarzy w kodzie programu,
- definiowanie i wykorzystywanie zmiennych,
- rozpoznawanie podstawowych typów danych,
- wyświetlanie informacji w terminalu za pomocą funkcji **`print()`**,
- pobieranie danych od użytkownika za pomocą funkcji **`input()`**,
- wykonywanie konwersji podstawowych typów danych,
- stosowanie podstawowych operatorów arytmetycznych,
- stosowanie operatorów porównania,
- stosowanie operatorów logicznych,
- tworzenie instrukcji warunkowych **`if`**,
- tworzenie instrukcji **`if...else`**,
- tworzenie instrukcji **`if...elif...else`**,
- stosowanie prawidłowych wcięć w kodzie programu,
- analizowanie działania programu dla różnych danych wejściowych,
- identyfikowanie podstawowych błędów składniowych i logicznych.

Politechnika Morska w Szczecinie. Wszelkie prawa autorskie zastrzeżone.

2. CZĘŚĆ TEORETYCZNA

2.1. Pierwszy program w języku Python

Python jest językiem programowania wysokiego poziomu, którego składnia została zaprojektowana w sposób ułatwiający tworzenie czytelnego kodu.

Program w języku Python może zostać zapisany w pliku posiadającym rozszerzenie:

```
.py
```

Przykładowa nazwa pliku:

```
laboratorium_L1.py
```

Najprostszy program może składać się z jednej instrukcji:

```
print("Hello World")
```

Po uruchomieniu programu w terminalu zostanie wyświetlony tekst:

```
Hello World
```

Funkcja `print()` umożliwia wyświetlanie tekstu, wartości zmiennych oraz wyników obliczeń.

Przykład:

```
print("Laboratorium L1")
print("Podstawy języka Python")
```


Politechnika Morska w Szczecinie. Wszelkie prawa autorskie zastrzeżone.

Komentarze w języku Python rozpoczynają się znakiem #

Przykład:

```
# To jest komentarz print("Hello World")
```

Komentarze nie są wykonywane przez interpreter. Służą przede wszystkim do opisywania działania programu oraz zwiększania jego czytelności.

2.2. Zmienne i podstawowe typy danych

Zmienna jest nazwanym miejscem wykorzystywanym do przechowywania danych.

Przykład:

```
wiek = 20
temperatura = 36.6
imie = "Jan"
aktywny = True
```

W przedstawionym przykładzie wykorzystano cztery podstawowe typy danych:

```
int    -  liczba całkowita
float  -  liczba zmiennoprzecinkowa
str    -  tekst
bool   -  wartość logiczna
```

Typ zmiennej można sprawdzić za pomocą funkcji:

```
type()
```

Przykład:

```
wiek = 20
```


Politechnika Morska w Szczecinie. Wszelkie prawa autorskie zastrzeżone.

```
print(type(wiek))
```

Wynik:

```
<class 'int'>
```

Analogicznie:

```
temperatura = 36.6
imie = "Jan"
aktywny = True

print(type(temperatura))
print(type(imie))
print(type(aktywny))
```

2.3. Zasady nazywania zmiennych

Nazwy zmiennych powinny być jednoznaczne i czytelne.

Poprawne przykłady:

```
temperatura = 25
temperatura_silnika = 90
napiecie_akumulatora = 12.5
predkosc_obrotowa = 1500
```

Nazwa zmiennej:

- **musi rozpoczynać się od litery lub znaku _,**
- **może zawierać litery, cyfry oraz znak _,**
- **nie może zawierać spacji,**
- **nie może rozpoczynać się cyfrą,**
- **nie może być słowem kluczowym języka Python.**

Politechnika Morska w Szczecinie. Wszelkie prawa autorskie zastrzeżone.

Poprawne:

```
x
liczba1
temperatura_silnika
_dane
```

Niepoprawne:

```
1liczba
temperatura silnika
@wartosc
```

Python rozróżnia wielkie i małe litery.

```
temperatura = 20
Temperatura = 50
```

Powyższe instrukcje tworzą dwie różne zmienne.

2.4. Funkcja `print()`

Funkcja `print()` służy do wyświetlania danych w terminalu.

Przykład:

```
print("Python")
```

Możliwe jest również wyświetlenie wartości zmiennej:

```
temperatura = 25
print(temperatura)
```


Politechnika Morska w Szczecinie. Wszelkie prawa autorskie zastrzeżone.

Tekst i zmienną można połączyć:

```
temperatura = 25
print("Temperatura wynosi:", temperatura)
```

Wynik:

```
Temperatura wynosi: 25
```

2.5. Pobieranie danych za pomocą `input()`

Funkcja **`input()`** umożliwia pobranie danych wprowadzonych przez użytkownika.

Przykład:

```
imie = input("Podaj swoje imię: ")
print("Witaj", imie)
```

Przykładowe działanie:

```
Podaj swoje imię: Jan
Witaj Jan
```

Wartość wprowadzona przez użytkownika zostaje zapisana w zmiennej i może zostać wykorzystana w dalszej części programu.

2.6. Konwersja typów danych

Funkcja `input()` zwraca wartość typu `str`.

Przykład:

```
wiek = input("Podaj wiek: ")
print(type(wiek))
```


Politechnika Morska w Szczecinie. Wszelkie prawa autorskie zastrzeżone.

Nawet jeżeli użytkownik wprowadzi `20` zmienna `wiek` będzie typu tekstowego.

Jeżeli wartość ma zostać wykorzystana jako liczba całkowita, należy zastosować:

```
int()
```

Przykład:

```
wiek = int(input("Podaj wiek: "))
```

Dla liczby zmiennoprzecinkowej:

```
temperatura = float(input("Podaj temperaturę: "))
```

Do podstawowych funkcji konwersji należą:

```
int()
float()
str()
bool()
```

Przykład:

```
x = "10"
x = int(x)
print(type(x))
```

2.7. Podstawowe operatory arytmetyczne

Python umożliwia wykonywanie podstawowych działań matematycznych.

```
+      dodawanie
-      odejmowanie
*      mnożenie
```


Politechnika Morska w Szczecinie. Wszelkie prawa autorskie zastrzeżone.

```
/     dzielenie
//    dzielenie całkowite
%     reszta z dzielenia
**    potęgowanie
```

Przykład:

```
a = 10
b = 4print(a + b)
print(a - b)
print(a * b)
print(a / b)
print(a // b)
print(a % b)
print(a ** b)
```

2.8. Operatory porównania

Instrukcje warunkowe wykorzystują warunki logiczne.

Do ich budowy stosowane są operatory porównania:

```
==    równe
!=    różne
>     większe
<     mniejsze
>=    większe lub równe
<=    mniejsze lub równe
```

Przykład:

```
x = 10
print(x > 5)
print(x == 10)
```


Politechnika Morska w Szczecinie. Wszelkie prawa autorskie zastrzeżone.

```
print(x < 3)
```

Wynik:

```
True
True
False
```

Należy zwrócić szczególną uwagę na różnicę pomiędzy = oraz ==

Operator = służy do przypisania wartości: x = 5

Operator == służy do porównania dwóch wartości x == 5

2.9. Operatory logiczne

W przypadku bardziej rozbudowanych warunków można stosować operatory logiczne:

```
and
or
not
```

Operator and wymaga spełnienia obu warunków:

```
temperatura = 75
wynik = temperatura > 50 and temperatura < 100
print(wynik)
```

Operator or wymaga spełnienia co najmniej jednego warunku:

```
x = 5
wynik = x < 0 or x > 3
print(wynik)
```

Operator not odwraca wartość logiczną:

Politechnika Morska w Szczecinie. Wszelkie prawa autorskie zastrzeżone.

```
stan = True
print(not stan)
```

2.10. Instrukcja warunkowa `if`

Instrukcja `if` służy do podejmowania decyzji w programie. Umożliwia wykonanie określonego fragmentu kodu tylko wtedy, gdy wskazany warunek ma wartość True. Taką rolę instrukcji `if` przedstawiono również w materiałach wykładowych. Podstawy informatyki

Ogólna składnia:

```
if warunek:
     instrukcja
```

Przykład:

```
temperatura = 90
if temperatura > 80:
    print("UWAGA! Wysoka temperatura")
```

Jeżeli warunek:

```
temperatura > 80
```

jest prawdziwy, wykonywana jest instrukcja znajdująca się wewnątrz bloku `if`.

2.11. Instrukcja `if...else`

Instrukcja `else` określa działanie programu w przypadku niespełnienia warunku.

```
if warunek:
    instrukcja_1
else:
```


Politechnika Morska w Szczecinie. Wszelkie prawa autorskie zastrzeżone.

```
    instrukcja_2
```

Przykład:

```
napiecie = float(input("Podaj napięcie [V]: "))
if napiecie >= 12:
    print("Napięcie prawidłowe")
else:
    print("Napięcie zbyt niskie")
```

W przedstawionym przypadku zawsze zostanie wykonany jeden z dwóch bloków kodu.

2.12. Instrukcja `if...elif...else`

W przypadku konieczności rozpatrzenia większej liczby stanów można zastosować instrukcję `elif`.

```
if warunek_1:
    instrukcja_1
elif warunek_2:
    instrukcja_2
else:
    instrukcja_3
```

Przykład:

```
temperatura = float(input("Podaj temperaturę [°C]: "))
if temperatura > 100:
    print("ALARM – temperatura krytyczna")
elif temperatura > 80:
    print("UWAGA – temperatura podwyższona")
else:
    print("Temperatura prawidłowa")
```


Politechnika Morska w Szczecinie. Wszelkie prawa autorskie zastrzeżone.

Warunki są sprawdzane kolejno od góry.

Po znalezieniu pierwszego warunku o wartości True wykonywany jest odpowiadający mu blok kodu, a pozostałe warunki w tej konstrukcji nie są już sprawdzane.

2.13. Wcięcia w języku Python

Wcięcia są elementem składni języka Python.

Przykład:

```
x = 10
if x > 5:
    print("x jest większe od 5")
print("Koniec programu")
```

Instrukcja:

```
print("x jest większe od 5")
```

należy do bloku `if`, ponieważ posiada wcięcie.

Instrukcja: `print("Koniec programu")` znajduje się poza blokiem warunkowym.

Zaleca się stosowanie czterech spacji jako jednego poziomu wcięcia.

Brak prawidłowego wcięcia może spowodować wystąpienie błędu **`IndentationError`**

2.14. Najczęściej popełniane błędy

Użycie = zamiast ==

Niepoprawnie:

```
if x = 5:
```


Politechnika Morska w Szczecinie. Wszelkie prawa autorskie zastrzeżone.

```
    print("x jest równe 5")
```

Poprawnie:

```
if x == 5:
    print("x jest równe 5")
```

Brak dwukropka

Niepoprawnie:

```
if x > 5
    print("x jest większe od 5")
```

Poprawnie:

```
if x > 5:
    print("x jest większe od 5")
```

Brak wcięcia

Niepoprawnie:

```
if x > 5:
print("x jest większe od 5")
```

Poprawnie:

```
if x > 5:
    print("x jest większe od 5")
```

Brak konwersji danych pobranych za pomocą `input()`

Niepoprawnie:

Politechnika Morska w Szczecinie. Wszelkie prawa autorskie zastrzeżone.

```
wiek = input("Podaj wiek: ")
if wiek >= 18:
     print("Osoba pełnoletnia")
```

Poprawnie:

```
wiek = int(input("Podaj wiek: "))
if wiek >= 18:
    print("Osoba pełnoletnia")
```


Politechnika Morska w Szczecinie. Wszelkie prawa autorskie zastrzeżone.

3. PRZEBIEG ĆWICZENIA

Zadania należy wykonywać kolejno. Każdy utworzony program należy uruchomić oraz sprawdzić jego działanie dla różnych danych wejściowych.

Zadanie 1 – pierwszy program

Cel: zapoznanie z podstawową strukturą programu oraz funkcją `print()`.

Utwórz program wyświetlający:

```
Laboratorium L1
Informatyka i języki programowania
Podstawy języka Python
```

Następnie utwórz zmienne:

```
imie = "Jan"
wiek = 20
```

i wyświetl komunikat:

```
Nazywam się Jan i mam 20 lat.
```

Sprawdź typ obu zmiennych przy użyciu funkcji `type()`.

Pytania

- Jakiego typu jest zmienna `imie`?
- Jakiego typu jest zmienna `wiek`?
- Jak zmieni się typ zmiennej po zapisaniu `wiek = "20"`?

Politechnika Morska w Szczecinie. Wszelkie prawa autorskie zastrzeżone.

Zadanie 2 – komunikacja z użytkownikiem

Napisz program, który:

- pobiera imię użytkownika,
- pobiera jego wiek,
- wyświetla pobrane informacje.

Przykładowy wynik:

```
Podaj imię: Jan
Podaj wiek: 20

Witaj Jan!
Masz 20 lat.
```

Sprawdź za pomocą funkcji `type()`, jakiego typu jest wartość zwracana przez `input()`. Następnie dokonaj konwersji wieku do typu `int`.

Politechnika Morska w Szczecinie. Wszelkie prawa autorskie zastrzeżone.

Zadanie 3 – prosty kalkulator

Napisz program pobierający od użytkownika dwie liczby a oraz b.

Program powinien obliczyć:

- sumę,
- różnicę,
- iloczyn,
- iloraz.

Wyniki zapisz do oddzielnych zmiennych i wyświetl w terminalu.

Przykład:

```
Podaj a: 10
Podaj b: 4

Suma: 14
Różnica: 6
Iloczyn: 40
Iloraz: 2.5
```

Pytanie

Co stanie się podczas próby dzielenia przez zero?

Politechnika Morska w Szczecinie. Wszelkie prawa autorskie zastrzeżone.

Zadanie 4 – instrukcja `if`

Napisz program pobierający temperaturę urządzenia.

Jeżeli `temperatura > 80°C`

program powinien wyświetlić:

```
UWAGA! Przekroczono dopuszczalną temperaturę.
```

Przetestuj program dla:

```
60°C
80°C
95°C
```

Pytania

- Dlaczego dla `80°C` komunikat nie jest wyświetlany?
- Jak należy zmienić warunek, aby wartość `80°C` również powodowała wyświetlenie ostrzeżenia?

Politechnika Morska w Szczecinie. Wszelkie prawa autorskie zastrzeżone.

Zadanie 5 – instrukcja `if...else`

Napisz program kontrolujący wartość napięcia.

Jeżeli `U >= 12 V`

program powinien wyświetlić:

```
Napięcie prawidłowe.
```

W przeciwnym przypadku:

```
UWAGA! Napięcie zbyt niskie.
```

Program należy przetestować dla co najmniej trzech wartości napięcia.

Politechnika Morska w Szczecinie. Wszelkie prawa autorskie zastrzeżone.

Zadanie 6 – instrukcja `if...elif...else`

Napisz program klasyfikujący temperaturę urządzenia według następujących kryteriów:

```
T > 100°C       ALARM – temperatura krytyczna
T > 80°C        UWAGA – temperatura podwyższona
pozostałe       Temperatura prawidłowa
```

Przetestuj program dla:

```
70°C
90°C
110°C
```

Pytania

- Jaki komunikat zostanie wyświetlony dla `100°C`?
- Jaki komunikat zostanie wyświetlony dla `80°C`?
- Dlaczego kolejność warunków ma znaczenie?

Politechnika Morska w Szczecinie. Wszelkie prawa autorskie zastrzeżone.

Zadanie 7 – zadanie samodzielne

System kontroli prędkości obrotowej

Napisz program pobierający od użytkownika wartość prędkości obrotowej:

```
n [obr/min]
```

Program powinien sklasyfikować stan pracy według następujących kryteriów:

```
n > 5000        Prędkość krytyczna
n > 3000        Wysoka prędkość
n > 0           Praca prawidłowa
pozostałe       Silnik zatrzymany
```

Program należy przetestować dla wartości reprezentujących wszystkie możliwe stany.

W kodzie należy zastosować:

```
input()
int() lub float()
if
elif
else
print()
```


Politechnika Morska w Szczecinie. Wszelkie prawa autorskie zastrzeżone.

4. WARUNKI ZALICZENIA

Warunkiem zaliczenia laboratorium jest wykonanie wskazanych zadań oraz wykazanie znajomości zastosowanych instrukcji.

Student powinien potrafić:

- wyjaśnić działanie utworzonego kodu,
- wskazać dane wejściowe i wynik programu,
- wyjaśnić zastosowane warunki,
- samodzielnie zmodyfikować wskazany fragment programu,
- przeanalizować działanie programu dla zmienionych danych wejściowych.

Do sprawozdania należy dołączyć zadania wskazane przez prowadzącego.

Politechnika Morska w Szczecinie. Wszelkie prawa autorskie zastrzeżone.

4.1 Wymagania formalne sprawozdania

Format i wysyłka

Sprawozdanie należy przygotować w formie **pliku PDF** i przesłać drogą mailową na adres:

l.mozga@pm.szczecin.pl

Temat wiadomości e-mail

Temat wiadomości musi mieć formę:

Imię_Nazwisko_NrAlbumu_Laboratorium_Lx

gdzie:

- Imię – imię studenta,
- Nazwisko – nazwisko studenta,
- NrAlbumu – numer albumu,
- Lx – numer laboratorium.

Przykład:
Jan_Kowalski_12345_Laboratorium_L12

Politechnika Morska w Szczecinie. Wszelkie prawa autorskie zastrzeżone.

Strona tytułowa sprawozdania

Na pierwszej stronie należy umieścić tabelę:

		Politechnika Morska w Szczecinie	
Laboratorium – Informatyka i języki programowania			
Temat :			
Imię i Nazwisko:		Prowadzący:	
Nr Albumu:			
Kierunek oraz rok:			
Data laboratorium:		Ocena:	

Zawartość sprawozdania

Sprawozdanie musi zawierać:

1. Stronę tytułową (tabela).
2. Treść zadania
3. Kod programu dla każdego zadania.
4. Uzyskane wyniki.
5. Wygenerowane wykresy (jeżeli wymaga treść zadania).
6. Krótkie wnioski (1–3 zdania do każdego zadania).

Brak któregokolwiek z powyższych elementów skutkuje brakiem zaliczenia laboratorium.

Politechnika Morska w Szczecinie. Wszelkie prawa autorskie zastrzeżone.

5. EFEKTY KSZTAŁCENIA

W wyniku realizacji laboratorium student:

1. tworzy i uruchamia prosty program w języku Python,
2. definiuje zmienne i rozpoznaje podstawowe typy danych,
3. stosuje funkcje `print()` oraz `input()`,
4. wykonuje podstawową konwersję typów danych,
5. wykorzystuje podstawowe operatory arytmetyczne,
6. wykorzystuje operatory porównania,
7. wykorzystuje operatory logiczne,
8. tworzy instrukcje `if`, `if...else` oraz `if...elif...else`,
9. stosuje prawidłowe wcięcia w kodzie,
10. analizuje działanie programu dla różnych danych wejściowych,
11. identyfikuje podstawowe błędy składniowe i logiczne,
12. interpretuje wyniki działania programu.

Politechnika Morska w Szczecinie. Wszelkie prawa autorskie zastrzeżone.

6. LITERATURA

1. E. Matthes, *Python. Instrukcje dla programisty*, Helion.
2. M. Lutz, *Python. Wprowadzenie*, Helion.
3. Oficjalna dokumentacja języka Python, Python Software Foundation.
4. Materiały wykładowe do przedmiotu *Informatyka i języki programowania*.

Politechnika Morska w Szczecinie. Wszelkie prawa autorskie zastrzeżone.

UWAGA KOŃCOWA

Programowanie należy poznawać poprzez samodzielne wykonywanie i modyfikowanie kodu.

W przypadku braku pewności dotyczącej działania określonego fragmentu programu należy:

1. przewidzieć wynik,
2. uruchomić kod,
3. sprawdzić otrzymany rezultat,
4. przeanalizować różnicę pomiędzy wynikiem oczekiwanym i rzeczywistym.

Błędy powstające podczas tworzenia programu są naturalnym elementem nauki programowania. Ważną umiejętnością jest nie tylko napisanie poprawnego kodu, lecz również rozpoznanie przyczyny błędu oraz jego samodzielne usunięcie.

Politechnika Morska w Szczecinie. Wszelkie prawa autorskie zastrzeżone.